

Análise da Execução e Identificação da Falha

Ao executar a função `soma_quebrada(10)`, o programa interrompe sua execução durante a primeira iteração do laço `while`, levantando um `AssertionError` na asserção de manutenção do invariante:

```
assert s == soma_esperada(i) and 0 <= i <= n
```

Essa asserção verifica se o invariante continua verdadeiro após a execução do corpo do laço.

Verificação da Primeira Iteração

Antes de entrar no laço:

```
s = 0  
i = 0
```

O invariante é satisfeito, pois $s = 0$ e $i(i + 1)/2 = 0$.

Na primeira iteração, o código executa:

```
s = s + i  
i = i + 1
```

Substituindo os valores:

```
s = 0 + 0 = 0  
i = 1
```

Após a atualização, a asserção verifica:

```
s == soma_esperada(1)
```

Mas $s = 0$ e $soma_esperada(1) = 1$. Como $0 \neq 1$, a asserção falha e gera o erro.

Razão Aritmética do Bug

O algoritmo deveria calcular:

$$1 + 2 + 3 + \cdots + n$$

Entretanto, ele calcula:

$$0 + 1 + 2 + \cdots + (n - 1)$$

Portanto, produz:

$$\frac{(n - 1)n}{2}$$

em vez de:

$$\frac{n(n + 1)}{2}$$

A diferença entre os dois resultados é exatamente n .

Por exemplo, para $n = 10$:

Resultado obtido: 45
Resultado correto: 55
Diferença: 10

Interpretação Geométrica do Erro

A soma dos n primeiros números naturais pode ser representada por um triângulo de pontos. Para $n = 5$, o triângulo correto possui:

$$1 + 2 + 3 + 4 + 5 = 15$$

pontos.

O algoritmo quebrado constrói:

$$0 + 1 + 2 + 3 + 4 = 10$$

pontos.

Isso significa que a última linha do triângulo, contendo n elementos, nunca é adicionada. Assim, o algoritmo gera um triângulo de altura $n - 1$ em vez de um triângulo de altura n , faltando exatamente n elementos.

Conclusão

A falha ocorre na asserção de manutenção do invariante, logo após a primeira iteração do laço. O erro lógico consiste em somar o valor atual de i antes de avançar para o próximo natural, fazendo com que o algoritmo calcule a soma de 0 até $n - 1$ em vez da soma de 1 até n .